Grupo 26		*Tema: 9*	
51938	*Tatiana Martins*	*90142*	*Bernardo Cozac*
90147	*Diogo Freitas*	*90247*	*Diogo Carvalho*

Clube LEI

O Clube LEI é uma associação desportiva dedicada à prática de ténis e padel, aberta a sócios e visitantes. Para além das suas atividades desportivas, o clube possui duas áreas de funcionamento complementares, a loja de artigos desportivos e o bar/cafetaria, que desempenham um papel central no apoio aos praticantes e visitantes e representam uma importante fonte de receita.

A loja comercializa artigos especializados para a prática de ténis e padel, como raquetes, bolas, calçado, calções, t-shirts, bonés, grips e sacos. Cada produto é caracterizado por um conjunto de atributos essenciais, nomeadamente: nome do artigo, categoria (ex.: raquete, vestuário, acessório), marca, modelo, preço de venda, código identificador único e quantidade em stock. O processo de venda está disponível para sócios e não sócios, sendo que os sócios beneficiam de um desconto mediante validação do número de sócio. Cada operação de venda deve registar dados fundamentais como a data e hora da compra, a lista de produtos vendidos e respetivas quantidades, o desconto aplicado, o meio de pagamento (ex.: numerário, cartão) e a identificação do funcionário responsável. A gestão de stock da loja inclui o acompanhamento dos níveis de cada produto e a definição de um valor mínimo por artigo. Quando o stock atinge esse valor, é realizada uma encomenda a fornecedores, operação que deve registar dados como produto, quantidade encomendada, fornecedor e funcionário que a emitiu.

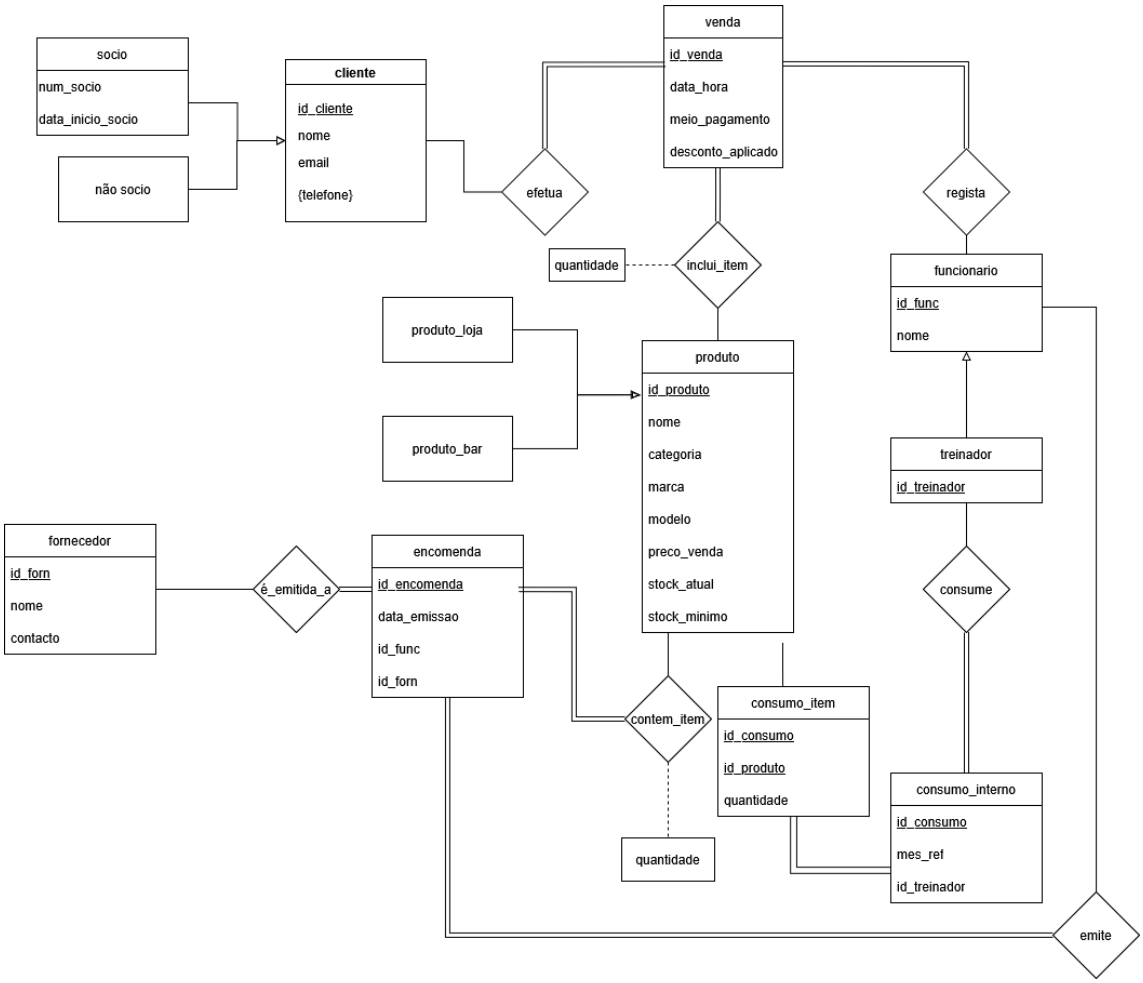
Para além das vendas, a loja também gere o consumo interno de bolas de ténis para as aulas dadas pelos treinadores. Este processo, que decorre no início de cada mês, consiste na entrega de uma quantidade de bolas a cada treinador, de acordo com o plano de treinos. Este movimento deve ser registado com informação como o mês de referência, o nome do treinador e a quantidade de bolas, permitindo que o consumo interno seja abatido ao stock e contabilizado como custo do clube.

O bar/cafetaria disponibiliza produtos de cafetaria (cafés, sumos, bolos, sandes) e refeições rápidas, incluindo pratos do dia, que se distinguem pela sua rotatividade. Tal como na loja, os sócios beneficiam de descontos sobre o preço de venda. Cada operação de venda deve registar dados semelhantes aos da loja: data e hora, produtos consumidos, quantidades, aplicação de desconto e funcionário responsável. A gestão de stock do bar é menos detalhada do que na loja, focando-se sobretudo em produtos finais prontos a servir (bebidas, snacks, refeições) e em matérias-primas essenciais (ex.: café, leite, pão), que devem ser monitorizadas para garantir a reposição atempada.

Face a este enquadramento, torna-se evidente a necessidade de uma base de dados integrada que consolide toda a informação relativa ao funcionamento do clube. Esta base de dados deverá contemplar:

- Gestão de clientes (sócios e não sócios), incluindo identificação e estatuto de sócio;
- Gestão de produtos (loja e bar), com atributos como nome, categoria, preço, stock e fornecedor;
- Registo de vendas, com indicação de data, hora, produtos, quantidades, descontos aplicados, meio de pagamento e funcionário responsável;
- Registo de consumo interno, associado a treinador, mês e quantidade de bolas;
- Gestão de stock e respetivos níveis mínimos;
- Gestão de encomendas a fornecedores, com indicação de produto, quantidade e responsável.

Modelo Entidade-Associação (ER)



Restrições de integridade que não são captadas pelo modelo

O desconto de sócio só pode ser aplicado quando o cliente tem estatuto de sócio e um número de sócio válido na data da venda.

O preço unitário de cada linha de venda fica fixado no momento da compra. Em cada inclui_item grava-se o preço vigente do produto nessa data. Assim o histórico é imutável e os relatórios não mudam quando o catálogo é atualizado.

As quantidades em inclui_item e em contem_item têm de ser inteiros positivos. Não se aceitam zeros nem negativos.

O stock nunca pode ficar negativo. As operações de venda e de consumo interno só são validadas se, depois do movimento, o stock do produto continuar maior ou igual a zero.

Os campos temporais não podem apontar para o futuro. A data e hora da venda e a data de emissão da encomenda têm de ser iguais ou anteriores ao momento do registo.

Decisões de modelação

Usámos uma especialização em Produto para distinguir Produto loja e Produto bar, ficando os atributos no supertipo Produto. Isto evita a repetição e mantém o controlo de stock e de preços num único sítio.

Colocámos Treinador como especialização de Funcionário porque nem todos os funcionários dão treinos, sendo que esta separação torna a regra clara e facilita a validação do consumo interno.

Para simplificar o modelo, os itens de venda e de encomenda ficam como atributos das relações inclui_item e contem_item, só com a quantidade.

Em Produto considerámos a dupla marca e modelo como chave candidata além do identificador interno.

Modelo relacional

cliente(id_cliente, nome, email)

cliente_telefone(id_cliente, telefone)
FK: id_cliente → cliente.id_cliente

socio(id_cliente, num_socio, data_inicio_socio)
FK: id_cliente → cliente.id_cliente

funcionario(id_func, nome)

treinador(id_treinador, id_func)
FK: id_func → funcionario.id_func

fornecedor(id_forn, nome, contacto)

produto(id_produto, nome, categoria, marca, modelo, preco_venda, stock_atual, stock_minimo)

produto_loja(id_produto)
FK: id_produto → produto.id_produto

produto_bar(id_produto)
FK: id_produto → produto.id_produto

venda(id_venda, data_hora, meio_pagamento, desconto_aplicado, id_cliente, id_func)
FK: id_cliente → cliente.id_cliente
FK: id_func → funcionario.id_func

venda_item(id_venda, id_produto, quantidade, preco_unitario)
FK: id_venda → venda.id_venda
FK: id_produto → produto.id_produto

encomenda(id_encomenda, data_emissao, id_func, id_forn)
FK: id_func → funcionario.id_func
FK: id_forn → fornecedor.id_forn

encomenda_item(id_encomenda, id_produto, quantidade)
FK: id_encomenda → encomenda.id_encomenda
FK: id_produto → produto.id_produto

consumo_interno(id_consumo, mes_ref, id_treinador)
FK: id_treinador → treinador.id_treinador

consumo_item(id_consumo, id_produto, quantidade)
FK: id_consumo → consumo_interno.id_consumo
FK: id_produto → produto_loja.id_produto

Restrições de integridade

Existem várias restrições de integridade que asseguram a consistência e correção dos dados na base de dados. As principais são as seguintes:

- As quantidades registadas em itens de venda, encomenda e consumo interno devem ser inteiros positivos, não sendo permitidos valores nulos, zero ou negativos.
- O preço de venda de cada produto e o preço unitário registado em cada linha de venda devem ser superiores a zero. O stock atual e mínimo devem ser valores não negativos.
- O desconto aplicado em cada venda deve situar-se entre 0 e 100 de modo a permitir a percentagem de desconto.
- As datas e horas de vendas e encomendas não podem ser posteriores à data atual do sistema. O mês de referência de consumos internos não pode ser posterior ao mês atual.
- O desconto de sócio só pode ser aplicado se, na data da venda, o cliente tiver um número de sócio válido e ativo.
- Nenhuma operação de venda ou consumo interno pode fazer com que o stock de um produto fique negativo.
- Cada número de sócio deve ser único entre os sócios ativos.
- Cada produto deve ter uma combinação única de marca e modelo, evitando duplicações no catálogo.
- Para cada treinador e mês, deve existir no máximo um registo de consumo interno, que agrupa todos os produtos consumidos nesse período.
- Em cada registo de consumo interno, cada produto só pode aparecer uma vez.
- Apenas produtos da loja podem ser incluídos em consumos internos.
- O preço unitário registado em cada linha de venda deve ficar fixo no momento da compra, garantindo a integridade histórica quando o catálogo é atualizado.
- O valor total de uma venda deve corresponder à soma dos produtos das linhas (quantidade × preço unitário), tendo em conta o desconto aplicado.

Código

ddl.sql

```
CREATE TABLE cliente (
    id_cliente      INT PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    email           VARCHAR(255)
);

CREATE TABLE cliente_telefone (
    id_cliente      INT NOT NULL,
    telefone        VARCHAR(20) NOT NULL,
    PRIMARY KEY (id_cliente, telefone),
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);

CREATE TABLE socio (
    id_cliente          INT  PRIMARY KEY,
    num_socio           INT  NOT NULL,
    data_inicio_socio   DATE NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente),
    CONSTRAINT uq_socio_num_socio UNIQUE (num_socio)
);

CREATE TABLE funcionario (
    id_func INT PRIMARY KEY,
    nome    VARCHAR(100) NOT NULL
);

CREATE TABLE treinador (
    id_treinador    INT PRIMARY KEY,
    id_func         INT NOT NULL,
    FOREIGN KEY (id_func) REFERENCES funcionario(id_func),
    CONSTRAINT uq_treinador_id_func UNIQUE (id_func)
);

CREATE TABLE fornecedor (
    id_forn     INT PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    contacto    VARCHAR(100)
);

CREATE TABLE produto (
    id_produto      INT PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    categoria       VARCHAR(50) NOT NULL,
    marca           VARCHAR(50) NOT NULL,
    modelo          VARCHAR(50) NOT NULL,
    preco_venda     NUMERIC(10,2) NOT NULL CHECK (preco_venda > 0),
    stock_atual     INT NOT NULL CHECK (stock_atual >= 0),
    stock_minimo    INT NOT NULL CHECK (stock_minimo >= 0),
    CONSTRAINT uq_produto_marca_modelo UNIQUE (marca, modelo)
);
```

```
CREATE TABLE produto_loja (
    id_produto  INT PRIMARY KEY,
    FOREIGN KEY (id_produto) REFERENCES produto(id_produto)
);

CREATE TABLE produto_bar (
    id_produto  INT PRIMARY KEY,
    FOREIGN KEY (id_produto) REFERENCES produto(id_produto)
);

CREATE TABLE venda (
    id_venda            INT PRIMARY KEY,
    data_hora           TIMESTAMP NOT NULL,
    meio_pagamento      VARCHAR(30) NOT NULL,
    desconto_aplicado   NUMERIC(5,2) NOT NULL CHECK (desconto_aplicado >= 0
AND desconto_aplicado <= 100),
    id_cliente          INT NOT NULL,
    id_func             INT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente),
    FOREIGN KEY (id_func) REFERENCES funcionario(id_func)
);

CREATE TABLE venda_item (
    id_venda        INT NOT NULL,
    id_produto      INT NOT NULL,
    quantidade      INT NOT NULL CHECK (quantidade > 0),
    preco_unitario  NUMERIC(10,2) NOT NULL CHECK (preco_unitario > 0),
    PRIMARY KEY (id_venda, id_produto),
    FOREIGN KEY (id_venda) REFERENCES venda(id_venda),
    FOREIGN KEY (id_produto) REFERENCES produto(id_produto)
);

CREATE TABLE encomenda (
    id_encomenda    INT PRIMARY KEY,
    data_emissao    TIMESTAMP NOT NULL,
    id_func         INT NOT NULL,
    id_forn         INT NOT NULL,
    FOREIGN KEY (id_func) REFERENCES funcionario(id_func),
    FOREIGN KEY (id_forn) REFERENCES fornecedor(id_forn)
);

CREATE TABLE encomenda_item (
    id_encomenda    INT NOT NULL,
    id_produto      INT NOT NULL,
    quantidade      INT NOT NULL CHECK (quantidade > 0),
    PRIMARY KEY (id_encomenda, id_produto),
    FOREIGN KEY (id_encomenda) REFERENCES encomenda(id_encomenda),
    FOREIGN KEY (id_produto) REFERENCES produto(id_produto)
);

CREATE TABLE consumo_interno (
    id_consumo  INT PRIMARY KEY,
    mes_ref     DATE NOT NULL,
    id_treinador INT NOT NULL,
    FOREIGN KEY (id_treinador) REFERENCES treinador(id_treinador),
    CONSTRAINT uq_consumo_mes_treinador UNIQUE (mes_ref, id_treinador)
);
```

```
CREATE TABLE consumo_item (
    id_consumo  INT NOT NULL,
    id_produto  INT NOT NULL,
    quantidade  INT NOT NULL CHECK (quantidade > 0),
    PRIMARY KEY (id_consumo, id_produto),
    FOREIGN KEY (id_consumo) REFERENCES consumo_interno(id_consumo),
    FOREIGN KEY (id_produto) REFERENCES produto_loja(id_produto)
);

-- Função que atualiza o stock
CREATE OR REPLACE FUNCTION atualiza_stock()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE produto
    SET stock_atual = stock_atual - NEW.quantidade
    WHERE id_produto = NEW.id_produto;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para venda_item
CREATE TRIGGER trg_atualiza_stock_venda
AFTER INSERT ON venda_item
FOR EACH ROW EXECUTE FUNCTION atualiza_stock();

-- Trigger para consumo_item
CREATE TRIGGER trg_atualiza_stock_consumo
AFTER INSERT ON consumo_item
FOR EACH ROW EXECUTE FUNCTION atualiza_stock();


-- Função que confirma stock insuficiente
CREATE OR REPLACE FUNCTION verifica_stock()
RETURNS TRIGGER AS $$
DECLARE
    stock_disponivel INT;
BEGIN
    SELECT stock_atual INTO stock_disponivel
    FROM produto WHERE id_produto = NEW.id_produto;

    IF stock_disponivel < NEW.quantidade THEN
        RAISE EXCEPTION 'Stock insuficiente. Disponível: %, Pedido: %',
            stock_disponivel, NEW.quantidade;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para venda_item
CREATE TRIGGER trg_verifica_stock_venda
BEFORE INSERT ON venda_item
FOR EACH ROW EXECUTE FUNCTION verifica_stock();

-- Trigger para consumo_item
CREATE TRIGGER trg_verifica_stock_consumo
BEFORE INSERT ON consumo_item
```

```
FOR EACH ROW EXECUTE FUNCTION verifica_stock();

-- Função que alerta quando stock atinge mínimo
CREATE OR REPLACE FUNCTION alerta_stock_minimo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stock_atual <= NEW.stock_minimo THEN
        RAISE NOTICE 'ALERTA: Produto % atingiu stock mínimo! Atual: %,
Mínimo: %',
            NEW.nome, NEW.stock_atual, NEW.stock_minimo;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger que dispara após UPDATE stock_atual ON produto
CREATE TRIGGER trg_alerta_stock
AFTER UPDATE OF stock_atual ON produto
FOR EACH ROW EXECUTE FUNCTION alerta_stock_minimo();

-- Função que garante desconto apenas para sócios
CREATE OR REPLACE FUNCTION verifica_desconto_socio()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.desconto_aplicado > 0 THEN
        IF NOT EXISTS (SELECT 1 FROM socio WHERE id_cliente = NEW.id_cliente)
THEN
            RAISE EXCEPTION 'Desconto só pode ser aplicado a sócios. Cliente %
não é sócio.',
                NEW.id_cliente;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para venda
CREATE TRIGGER trg_verifica_desconto
BEFORE INSERT ON venda
FOR EACH ROW EXECUTE FUNCTION verifica_desconto_socio();
```

dml insert.sql

```
INSERT INTO cliente (id_cliente, nome, email) VALUES
(1, 'João Silva', 'joao.silva@exemplo.pt'),
(2, 'Maria Oliveira', 'maria.oliveira@exemplo.pt'),
(3, 'Ricardo Costa', 'ricardo.costa@exemplo.pt'),
(4, 'Ana Rodrigues', 'ana.rodrigues@exemplo.pt'),
(5, 'Tiago Fernandes', 'tiago.fernandes@exemplo.pt'),
(6, 'Sofia Santos', 'sofia.santos@exemplo.pt'),
(7, 'Pedro Carvalho', 'pedro.carvalho@exemplo.pt'),
(8, 'Inês Marques', 'ines.marques@exemplo.pt'),
(9, 'Luís Gonçalves', 'luis.goncalves@exemplo.pt'),
(10,'Carla Nogueira','carla.nogueira@exemplo.pt');

INSERT INTO cliente_telefone (id_cliente, telefone) VALUES
(1, '912345678'),
(1, '965432100'),
(2, '913001001'),
(3, '914112233'),
(4, '915223344'),
(5, '916334455'),
(6, '917445566'),
(7, '918556677'),
(8, '919667788'),
(9, '911778899'),
(10, '919045896');

INSERT INTO socio (id_cliente, num_socio, data_inicio_socio) VALUES
(1, 1001, '2023-01-10'),
(2, 1002, '2023-02-15'),
(3, 1003, '2023-03-20'),
(4, 1004, '2023-04-05'),
(5, 1005, '2023-05-12');


INSERT INTO funcionario (id_func, nome) VALUES
(1, 'Miguel Pereira'),
(2, 'Rita Almeida'),
(3, 'Nuno Figueiredo'),
(4, 'Helena Matos'),
(5, 'Bruno Lopes'),
(6, 'Carolina Freitas'),
(7, 'André Barbosa'),
(8, 'Filipa Sousa'),
(9, 'Hugo Machado'),
(10,'Patrícia Reis');

INSERT INTO treinador (id_treinador, id_func) VALUES
(1, 3),
(2, 5),
(3, 9),
(4, 4),
(5, 1);


INSERT INTO fornecedor (id_forn, nome, contacto) VALUES
```

```
(1, 'Fornecedora Atlântico, Lda', 'atlantic@fornecedores.pt'),
(2, 'NutriDesporto SA', 'nutri@fornecedores.pt'),
(3, 'Bebidas do Sul Lda', 'comercial@bebidassul.pt'),
(4, 'Equipamentos FitLife', 'vendas@fitlife.pt'),
(5, 'Saúde & Energia Lda', 'geral@saudeenergia.pt'),
(6, 'CoolDrinks Portugal', 'info@cooldrinks.pt'),
(7, 'Snacks Saudáveis SA', 'snacks@saudaveis.pt'),
(8, 'TechGym Iberia', 'contacto@techgym.pt'),
(9, 'Águas da Serra', 'comercial@aguaserrapt.pt'),
(10,'SportsWear Algarve', 'vendas@sportswearalgarve.pt');

INSERT INTO produto (id_produto, nome, categoria, marca, modelo, preco_venda,
stock_atual, stock_minimo) VALUES
(1, 'Garrafa de água 0.5L', 'bebida', 'Águas da Serra', 'Serra 0.5', 0.80,
200, 50),
(2, 'Barra proteica chocolate', 'snack', 'NutriBar', 'ChocoPro', 1.80, 150,
30),
(3, 'Barra proteica amendoim', 'snack', 'NutriBar', 'PeanutPro', 1.80, 120,
25),
(4, 'Bebida isotónica limão', 'bebida', 'IsoMax', 'Limao500', 1.50, 180, 40),
(5, 'Bebida isotónica frutos vermelhos', 'bebida', 'IsoMax', 'FrutosV500',
1.60, 160, 40),
(6, 'Toalha ginásio pequena', 'acessório', 'FitLife', 'TowelS', 5.50, 60, 10),
(7, 'Toalha ginásio grande', 'acessório', 'FitLife', 'TowelL', 8.90, 40, 10),
(8, 'T-shirt club homem', 'têxtil', 'SportsWear', 'TSH-H-2025', 12.90, 80,
20),
(9, 'T-shirt club mulher', 'têxtil', 'SportsWear', 'TSH-M-2025', 12.90, 70,
20),
(10,'Shaker proteína 600ml', 'acessório', 'NutriShaker', 'NS600', 6.50, 90,
15),
(11, 'Bolas de ténis (pack 4)', 'equipamento', 'Wilson', 'Championship', 8.50,
500, 100);

INSERT INTO produto_loja (id_produto) VALUES
(6),
(7),
(8),
(9),
(11);

INSERT INTO produto_bar (id_produto) VALUES
(1),
(2),
(3),
(4),
(5),
(10);

INSERT INTO venda (id_venda, data_hora, meio_pagamento, desconto_aplicado,
id_cliente, id_func) VALUES
(1,  '2025-01-10 09:15:00', 'dinheiro',   5.00,  1,  10),
(2,  '2025-01-10 10:30:00', 'multibanco', 5.00,  2,  2),
(3,  '2025-01-11 11:05:00', 'mbway',      10.00, 3,  10),
(4,  '2025-01-11 17:20:00', 'multibanco', 10.00, 4,  7),
(5,  '2025-01-12 08:50:00', 'dinheiro',   5.00,  5,  2),
(6,  '2025-01-12 19:10:00', 'mbway',      0.00,  6,  6),
(7,  '2025-01-13 09:40:00', 'multibanco', 0.00,  7,  6),
(8,  '2025-01-13 18:25:00', 'dinheiro',   0.00,  8,  8),
(9,  '2025-01-14 12:15:00', 'multibanco', 0.00,  9,  2),
```

```
(10, '2025-01-14 20:05:00', 'mbway',      0.00,  10, 7);

INSERT INTO venda_item (id_venda, id_produto, quantidade, preco_unitario)
VALUES
(1, 1, 2, 0.80),
(1, 2, 1, 1.80),
(2, 4, 2, 1.50),
(3, 2, 1, 1.80),
(4, 6, 1, 5.50),
(5, 1, 1, 0.80),
(6, 3, 2, 1.80),
(7, 8, 1, 12.90),
(8, 9, 1, 12.90),
(9, 10,1, 6.50),
(10, 3,1, 1.80);

INSERT INTO encomenda (id_encomenda, data_emissao, id_func, id_forn) VALUES
(1, '2024-12-01 10:00:00', 7, 9),
(2, '2024-12-02 11:30:00', 8, 2),
(3, '2024-12-03 09:45:00', 7, 7),
(4, '2024-12-04 14:10:00', 2, 3),
(5, '2024-12-05 15:20:00', 2, 4),
(6, '2024-12-06 16:05:00', 6, 1),
(7, '2024-12-07 10:50:00', 6, 5),
(8, '2024-12-08 13:15:00', 7, 6),
(9, '2024-12-09 09:05:00', 8, 8),
(10,'2024-12-10 17:40:00', 10,10);

INSERT INTO encomenda_item (id_encomenda, id_produto, quantidade) VALUES
(1, 1, 300),
(2, 2, 200),
(3, 3, 150),
(4, 4, 180),
(5, 5, 160),
(6, 6, 80),
(7, 7, 60),
(8, 8, 100),
(9, 9, 90),
(10,10,120);

INSERT INTO consumo_interno (id_consumo, mes_ref, id_treinador) VALUES
(1, '2025-01-01', 1),
(2, '2025-02-01', 2),
(3, '2025-03-01', 3),
(4, '2025-04-01', 4),
(5, '2025-05-01', 5),
(6, '2025-06-01', 1),
(7, '2025-07-01', 2),
(8, '2025-08-01', 3),
(9, '2025-09-01', 4),
(10,'2025-10-01',5);

INSERT INTO consumo_item (id_consumo, id_produto, quantidade) VALUES
(1, 11, 10),
(2, 11, 8),
(3, 11, 6),
(4, 11, 5),
```

```
(5, 11, 7),
(6, 11, 6),
(7, 11, 4),
(8, 11, 5),
(9, 11, 3),
(10, 11, 4);
```

queries.sql

```
-- Q1: Obter as encomendas realizadas pela funcionária Rita Almeida
SELECT e.id_encomenda, e.data_emissao, f.nome
FROM encomenda e, funcionario f
WHERE e.id_func = f.id_func
  AND f.nome = 'Rita Almeida';


-- Q2: Obter o total de vendas por cliente que é sócio
SELECT c.nome, COUNT(v.id_venda) AS total_vendas
FROM cliente c, venda v, socio s
WHERE c.id_cliente = v.id_cliente
  AND c.id_cliente = s.id_cliente
GROUP BY c.nome;


-- Q3: Obter os produtos com mais de 2 unidades vendidas no total
SELECT p.nome, SUM(vi.quantidade) AS total_vendido
FROM produto p, venda_item vi
WHERE p.id_produto = vi.id_produto
GROUP BY p.nome
HAVING SUM(vi.quantidade) > 2;


-- Q4: Obter todos os funcionários e indicar se são treinadores
SELECT f.nome, t.id_treinador
FROM funcionario f
LEFT JOIN treinador t ON f.id_func = t.id_func;


-- Q5: Obter os clientes que não são sócios
SELECT c.nome, c.email
FROM cliente c
WHERE c.id_cliente NOT IN (SELECT id_cliente FROM socio);

-- Q6: Alerta Stock Mínimo
WITH ProdutosCriticos AS (
    SELECT id_produto, nome, stock_atual, stock_minimo
    FROM produto
    WHERE stock_atual <= stock_minimo
)
SELECT nome, stock_atual, (stock_minimo - stock_atual) AS quantidade_em_falta
FROM ProdutosCriticos;

-- Q7: Obter o nome dos fornecedores que nunca receberam nenhuma encomenda do
clube.
SELECT f.nome
FROM fornecedor f
WHERE NOT EXISTS (
    SELECT 1
    FROM encomenda e
    WHERE e.id_forn = f.id_forn
);

-- Q.8: Encontrar pares de produtos diferentes que tenham exatamente o mesmo
preço de venda.
```

```
SELECT p1.nome AS produto_A, p2.nome AS produto_B, p1.preco_venda
FROM produto p1, produto p2
WHERE p1.preco_venda = p2.preco_venda
  AND p1.id_produto < p2.id_produto;

-- Q9: Obter o nome dos clientes que compraram TODOS os produtos da marca
'Wilson'.
-- (Lógica: Selecionar clientes para os quais NÃO EXISTE um produto 'Wilson'
que eles NÃO tenham comprado).
SELECT c.nome
FROM cliente c
WHERE NOT EXISTS (
    SELECT p.id_produto
    FROM produto p
    WHERE p.marca = 'Wilson'
      AND NOT EXISTS (
          SELECT v.id_venda
          FROM venda v, venda_item vi
          WHERE v.id_cliente = c.id_cliente
            AND v.id_venda = vi.id_venda
            AND vi.id_produto = p.id_produto
      )
);

-- Q.10: Obter o nome e categoria dos produtos que começam pela palavra
"Raquete".
SELECT nome, categoria
FROM produto
WHERE nome LIKE 'Raquete%';

-- Q.11: Obter os produtos que são mais caros do que todos os produtos da
categoria 'Acessório'.
SELECT nome, preco_venda
FROM produto
WHERE preco_venda > ALL (
    SELECT preco_venda
    FROM produto
    WHERE categoria = 'Acessório'
);

-- Q.12: Calcular o valor total faturado por venda, listando a data, o nome do
cliente e o valor final a pagar (considerando a soma dos itens e aplicando o
desconto percentual da venda).
SELECT v.id_venda,
       v.data_hora,
       c.nome AS cliente,
       SUM(vi.quantidade * vi.preco_unitario) AS subtotal_bruto,
       v.desconto_aplicado,
       SUM(vi.quantidade * vi.preco_unitario) * (1 - (v.desconto_aplicado /
100.0)) AS valor_final_liquido
FROM venda v
JOIN cliente c ON v.id_cliente = c.id_cliente
JOIN venda_item vi ON v.id_venda = vi.id_venda
GROUP BY v.id_venda, v.data_hora, c.nome, v.desconto_aplicado
ORDER BY valor_final_liquido DESC;
```

proc_func.sql

```
--
================================================================================
=======
-- REQUISITO: Stored Procedure para dar resposta a um requisito funcional
--
================================================================================
=======
-- Descrição: O sistema de gestão de stock necessita de uma forma de dar
entrada
-- dos produtos quando uma encomenda chega do fornecedor.
-- Este procedimento percorre os itens de uma encomenda específica e soma as
-- quantidades recebidas ao stock atual dos respetivos produtos.

CREATE OR REPLACE PROCEDURE sp_processar_chegada_encomenda(p_id_encomenda INT)
LANGUAGE plpgsql
AS $$
DECLARE
    registo RECORD;
    encomenda_existe BOOLEAN;
BEGIN
    -- 1. Verificar se a encomenda existe
    SELECT EXISTS(SELECT 1 FROM encomenda WHERE id_encomenda = p_id_encomenda)
    INTO encomenda_existe;

    IF NOT encomenda_existe THEN
        RAISE EXCEPTION 'A encomenda % não existe.', p_id_encomenda;
    END IF;

    -- 2. Percorrer todos os itens da encomenda
    FOR registo IN
        SELECT id_produto, quantidade
        FROM encomenda_item
        WHERE id_encomenda = p_id_encomenda
    LOOP
        -- 3. Atualizar o stock do produto (Stock Atual + Quantidade da
Encomenda)
        UPDATE produto
        SET stock_atual = stock_atual + registo.quantidade
        WHERE id_produto = registo.id_produto;

        -- Log opcional (para debug)
        RAISE NOTICE 'Produto ID %: Stock aumentado em % unidades.',
registo.id_produto, registo.quantidade;
    END LOOP;

    RAISE NOTICE 'Encomenda % processada com sucesso. Stocks atualizados.',
p_id_encomenda;
END;
$$;

-- EXEMPLO DE UTILIZAÇÃO DO PROCEDURE:
-- Vamos simular a chegada da Encomenda nº 10 (que tem 120 unidades do produto
10).
-- O stock atual do produto 10 é 90 (ver dml_insert.sql).
-- Após correr este comando, o stock deverá passar para 210.
```

```
/*
CALL sp_processar_chegada_encomenda(10);
*/

--
================================================================================
=======
-- REQUISITO EXTRA (Função): Calcular total gasto por um cliente
--
================================================================================
=======
-- Descrição: Função auxiliar para calcular o valor total (líquido) gasto por
um cliente,
-- útil para determinar clientes VIP.

CREATE OR REPLACE FUNCTION fn_calcular_total_gasto_cliente(p_id_cliente INT)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    total_gasto NUMERIC(10,2);
BEGIN
    SELECT COALESCE(SUM(total_liquido), 0) INTO total_gasto
    FROM (
        SELECT
            SUM(vi.quantidade * vi.preco_unitario) * (1 - (v.desconto_aplicado
/ 100.0)) AS total_liquido
        FROM venda v
        JOIN venda_item vi ON v.id_venda = vi.id_venda
        WHERE v.id_cliente = p_id_cliente
        GROUP BY v.id_venda
    ) subquery;

    RETURN total_gasto;
END;
$$;
```

views.sql

```
--
================================================================================
=======
-- VISTA 1: Catálogo Público de Produtos
--
================================================================================
=======
-- Requisito/Justificação:
-- Os clientes e visitantes devem poder consultar os produtos e preços, mas
não devem
-- ter acesso a informações sensíveis de gestão, como a quantidade exata de
stock,
-- o stock mínimo ou o fornecedor.
-- Esta vista oculta o 'stock_atual' e substitui por um indicador de
disponibilidade.

CREATE OR REPLACE VIEW v_catalogo_publico AS
SELECT
    nome AS produto,
    categoria,
    marca,
    modelo,
    preco_venda AS preco,
    CASE
        WHEN stock_atual > 0 THEN 'Disponível'
        ELSE 'Esgotado'
    END AS estado_stock
FROM produto;

-- EXEMPLO DE UTILIZAÇÃO:
-- Consultar o catálogo de raquetes e bebidas disponíveis para venda.
/*
SELECT * FROM v_catalogo_publico
WHERE categoria IN ('bebida', 'equipamento');
*/

--
================================================================================
=======
-- VISTA 2: Lista de Contactos de Sócios Ativos
--
================================================================================
=======
-- Requisito/Justificação:
-- A receção necessita de uma lista rápida para contactar sócios, sem
necessitar de fazer
-- joins complexos manualmente entre as tabelas 'cliente', 'socio' e
'cliente_telefone'.
-- Esta vista filtra apenas quem é sócio e apresenta os dados de contacto
consolidados.

CREATE OR REPLACE VIEW v_contactos_socios AS
SELECT
    s.num_socio,
    c.nome,
    c.email,
    ct.telefone,
```

```
    s.data_inicio_socio
FROM socio s
JOIN cliente c ON s.id_cliente = c.id_cliente
LEFT JOIN cliente_telefone ct ON c.id_cliente = ct.id_cliente;
```